```
}
```

Note that the ts[] is an array, and D has automatically declared and initialized it for you. Setting the value of a variable to 0 instructs dtrace to recover the memory of the variable. The output gives the function name and the total time spent in that function during the time the script ran.

```
# ./myDscript.d 23
dtrace: script './myDscript' matched 15887 probes
^C
  enqueue                    2952
  mutex_lock                 3151262
  cond_timedwait             31282360711332
```

5.2.7 Ignoring Functions Already Entered

Notice that the cond_timedwait line in the previous example has a very large time number. This is because the cond_timedwait() function was already executing when the D script was started. To avoid this condition, add the following predicate to the return probe section. This predicate ignores a function return if there was no enter for that function.

```
pid$1:::return
/ts[probefunc] != 0/
{
          @func_time[probefunc] = sum(timestamp \
                    - ts[probefunc]);
          ts[probefunc] = 0;
}
```

As in C, you can omit the != 0 part and just use /ts[probefunc]/ as the predicate.

5.2.8 Handling Multithreaded Applications

Two threads could execute the same function at the same time and produce a race condition. To handle this case you need one copy of the ts[] construct for each thread. DTrace addresses this with the self variable. Anything that you add to the self variable is made thread local. The following script ignores functions that were already entered and handles multithreaded applications.

```
#!/usr/sbin/dtrace -s
pid$1:::entry
{
```